Flight Tracker

NestJS + Next.js Codebase Guide

For automation testers learning full-stack development

1 — Repository Layout

The project has three top-level items plus infrastructure config.

```
flight-tracker/
  backend/           NestJS server  (TypeScript, port 3100)
  frontend/          Next.js browser app  (TypeScript + React, port 3000)
  docker-compose.yml Starts PostgreSQL 16 + Redis 7
  README.md          Quick-start guide
```

2 — Technology Stack

Layer	Technology	Job
Frontend framework	Next.js 15	Routing, SSR, bundling
UI library	React 19	Component-based UI
Language (both)	TypeScript	Type-safe JavaScript
Styling	Tailwind CSS	Utility-first CSS
UI primitives	shadcn/ui + Radix	Accessible components
Map	react-leaflet	Interactive Leaflet map as React
Data fetching	fetch + swr	HTTP calls + caching hooks
Backend framework	NestJS + Express	Structured REST API
Database	PostgreSQL + TypeORM	Flight movement history
Cache	Redis + cache-manager	API response caching 15–30 min
Background jobs	BullMQ + @nestjs/bull	Airport history collector
Live positions	adsb.lol	Free ADS-B transponder network
Schedules/status	AeroDataBox (Pro)	Paid API — 5,000 units/month
Weather	Open-Meteo	Free hourly forecasts
Infrastructure	Docker + Compose	Local PostgreSQL + Redis

3 — Backend (backend/)

NestJS organises code into **Modules**, each containing a **Controller** (handles HTTP) and a **Service** (contains logic). Dependencies are injected automatically — like Playwright fixtures.

3.1 NestJS Core Concepts

Concept	What it does	Playwright analogy
Module	Groups controller + service + imports	Page Object file
Controller	Receives request, validates params, calls service	Test step calling a POM
Service	Business logic — knows nothing about HTTP	POM method body
DI	NestJS auto-creates services declared in constructors	Fixtures providing { page }
Entity	TypeScript class mapped to a DB table	Data schema / model
Repository	Read/write methods for one entity	Data-access layer

3.2 Module Map (backend/src/)

Module / File	HTTP endpoint(s)	Job
main.ts	—	Bootstrap: ValidationPipe, CORS, listen :3100
app.module.ts	—	Root: wires TypeORM, Redis, BullMQ, all feature modules
flight/	GET /api/aircraft /api/flight	Live aircraft near point; full flight detail + chain
route/	GET /api/search /api/route	Search by flight number; flights on a route+date
airport/	GET /api/delays	Airport arrivals/departures with delay rankings
airline/	GET /api/airlines	ICAO code → airline name map
analytics/	GET /api/history	Movement count summary from PostgreSQL
geo/	GET /api/place	Geocode place name → lat/lon
prediction/	—	Delay prediction engine (used by flight service)
movement/	—	TypeORM entity + repository for flight history
collector/	—	BullMQ processor: background airport board collection
sources/adsb/	—	adsb.lol client — live aircraft positions
sources/aerodatabox/	—	AeroDataBox client — schedules, status, gates
sources/airlines/	—	GitHub CSV → airline name lookup (cached 24h)
sources/weather/	—	Open-Meteo client (cached 30 min in Redis)
sources/geocode/	—	Open-Meteo geocoding (cached 1h in Redis)

3.3 Database (movement/)

PostgreSQL stores every observed flight movement. TypeORM maps the **Movement** entity (direction + flight number + scheduled time = composite PK) to a table. The repository's **saveMovements()** upserts rows but never overwrites a confirmed actual time with a null.

The prediction engine queries this table to find historical delay patterns for a given flight number.

3.4 Caching Strategy

Data	TTL	Where cached
AeroDataBox flight lookup	15 min	Redis
AeroDataBox airport board	15 min	Redis
AeroDataBox route search	30 min	Redis
Weather forecast	30 min	Redis
Geocode result	1 hour	Redis
Airline name CSV	24 hours	Redis
ADS-B live positions	30 sec	In-process memory

3.5 Background Collector (collector/)

A BullMQ repeating job (interval set by COLLECT_INTERVAL_HOURS) fetches the full arrival+departure board for every airport in COLLECT_AIRPORTS. Results are saved to PostgreSQL, building the historical delay dataset. A 1.5-second delay between airports prevents AeroDataBox rate limits.

COLLECT_AIRPORTS is unset by default to protect API quota. On the Pro plan (5,000 units/month) it is safe to enable.

4 — Frontend (frontend/)

A Next.js 15 app with the App Router. Three tabs — Map, Find Flight, Delays — each rendered as a React component. Tailwind CSS handles all styling via CSS-variable-based tokens for light/dark theming.

4.1 File Map (frontend/src/)

File / Folder	Job
app/layout.tsx	HTML skeleton, Inter font, global CSS import
app/page.tsx	Tab state, URL hash routing (#flight/6E5321, #delays)
app/globals.css	CSS variables for light/dark themes, altitude colour bands, Leaflet overrides
components/tracker/MapTab.tsx	Map tab: 10s refresh loop, aircraft sidebar, info panel
components/tracker/MapInner.tsx	react-leaflet map (SSR disabled — client only)
components/flight/FlightSearchTab.tsx	Find Flight tab: search form, flight card, timeline, chain
components/delays/DelaysTab.tsx	Delays tab: flight table with direction filter, airline ranking chart
lib/api.ts	All fetch calls to the backend — one function per endpoint
lib/types.ts	TypeScript interfaces matching backend response shapes
lib/format.ts	Format times, delays, durations into readable strings
lib/flight-status.ts	Pure logic: flight leg → { icon, text, tone, detail }

4.2 Map Tab

Fetches **GET /api/aircraft** every 10 seconds. Renders SVG plane icons on a Leaflet map coloured by altitude band (ground → cruise). Selecting an aircraft calls **GET /api/flight** and shows the full day chain with prediction badges.

Leaflet cannot run on the server, so MapInner is loaded with dynamic(import, { ssr: false }).

4.3 Find Flight Tab

Search by flight number (e.g. 6E 5321) or pair of airport codes. The flight card shows: status badge, scheduled vs actual times, gate, terminal, delay in minutes, and the aircraft\'s full day chain.

4.4 Delays Tab

Calls **GET /api/delays** for a chosen airport. Displays a ranked bar chart of airlines by average delay, and a filterable table of individual arrivals/departures.

4.5 Tailwind Theming

```
/* globals.css */
:root {
  --primary: 221 83% 53%;     /* blue */
  --background: 0 0% 100%;
}
.dark {
  --primary: 217 91% 60%;
  --background: 222 47% 11%;
```

```
}
```

5 — Infrastructure (docker-compose.yml)

Service	Port	Volume	Job
PostgreSQL 16	5432	pg_data	Flight movement history, delay analytics
Redis 7	6379	redis_data	API response cache + BullMQ job queue

```
docker compose up -d    # start both
docker compose down     # stop (data kept in volumes)
docker compose down -v  # stop AND delete all data
```

6 — Environment Variables (backend/.env)

Variable	Required	Default / Example	Purpose
PORT	no	3100	HTTP listen port
FRONTEND_URL	no	http://localhost:3000	CORS origin
DATABASE_URL	yes	postgresql://...@localhost/...	PostgreSQL connection string
REDIS_HOST	no	localhost	Redis host
REDIS_PORT	no	6379	Redis port
AERODATABOX_KEY	yes*	your RapidAPI key	Schedules, status, gates (* app runs without it, but only
ADSB_CONTACT	no	your-email@example.com	Sent in User-Agent to adsb.lol (their ToS)
COLLECT_AIRPORTS	no	BLR,DEL	Airports for background history collection
COLLECT_INTERVAL_HOURS	no	12	How often the collector runs

7 — API Endpoints

Method	Path	Key params	Returns
GET	/api/aircraft	lat, lon, dist (nm)	Array of live aircraft near the point
GET	/api/flight	callsign or hex	Full flight detail: schedule, chain, prediction, live position
GET	/api/search	number (e.g. '6E 5321')	Matching flight legs for today
GET	/api/route	origin, destination, date	All flights between two airports on a date
GET	/api/delays	icao (airport code)	Arrivals + departures with delay info, airline rankings
GET	/api/airlines	—	Map of ICAO code → airline name
GET	/api/history	—	Total movement count recorded in PostgreSQL
GET	/api/place	q (place name)	Geocoded lat/lon for a city or airport

8 — End-to-End Data Flow: Searching '6E 5321'

Step	What happens	Code location
1	User types '6E 5321', presses Search	FlightSearchTab.tsx
2	React calls api.search('6E 5321')	frontend/src/lib/api.ts
3	GET /api/search?number=6E%205321 sent to backend	HTTP fetch
4	RouteController receives and validates the param	route/route.controller.ts
5	RouteService.searchFlight() called	route/route.service.ts
6	Redis miss — AeroDataBox queried for today's flights	sources/aerodatabox/aerodatabox.service.ts
7	AeroDataBox returns schedule, gate, status	aerodatabox.service.ts → toLeg()
8	adsb.lol queried for live position	sources/adsb/adsb.service.ts
9	Both results upserted into PostgreSQL	movement/movement.repository.ts
10	JSON response cached in Redis for 15 min	NestJS CacheInterceptor
11	JSON returned to browser; React re-renders card	FlightSearchTab.tsx
12	flight-status.ts computes '■ Delayed +17 min'	frontend/src/lib/flight-status.ts

First request: 300–800 ms. Cache hit (within 15 min): <50 ms.

9 — Delay Prediction Engine (prediction/prediction.service.ts)

Core rule: **an aircraft cannot depart until it has arrived and completed its minimum turnaround.** The engine walks the aircraft\'s full day chain and propagates any knock-on delay forward.

```
Example chain:

  Leg 1: BOM → BLR  scheduled 06:00  actual arrival 08:15 (+45 min late)
  Leg 2: BLR → DEL  scheduled 08:30  (60 min scheduled turnaround)

  Minimum turnaround for narrow-body = 35 min
  Slack = 60 - 35 = 25 min
  Knock-on delay = 45 - 25 = +20 min
  Predicted departure = 08:50
```

Prediction inputs

Input	Source	Weight
Inbound flight actual/estimated arrival	AeroDataBox	Primary — determines knock-on
Aircraft type	ADS-B	Sets minimum turnaround (35–60 min)
Weather at origin + destination	Open-Meteo	Bumps risk to High in storms
Historical delay rate for this flight	PostgreSQL	Adds warning note if >40% of past flights late
Airline\'s revised estimate	AeroDataBox	Overrides calculation when available

Output per leg: **risk level** (Low / Moderate / High), **expected window** (e.g. 08:45–09:10), **reasons list** (inbound delay, turnaround, weather, history).

10 — Quick Start

```
# 1. Start PostgreSQL + Redis
docker compose up -d

# 2. Backend
cd backend
cp .env.example .env   # AERODATABOX_KEY already set
npm install
npm run start:dev      # http://localhost:3100

# 3. Frontend  (new terminal)
cd frontend
cp .env.example .env
npm install
npm run dev            # http://localhost:3000
```

11 — Connecting to Your Playwright Knowledge

Playwright concept	Equivalent in this app

page.goto('http://...')	Browser loading the Next.js app at localhost:3000
page.request.get('/api/...')	Frontend fetch calls to the NestJS backend at :3100
Test fixtures / beforeAll setup	docker compose up -d (starts PostgreSQL + Redis)
TypeScript types in tests	TypeScript types in both backend and frontend
Page Object Model	React components — each tab is a self-contained component
network mocking (page.route())	Test mocks replacing AeroDataBox with fake responses
test.use({ baseURL })	NEXT_PUBLIC_API_URL env var pointing to the backend
Assertions on page state	What you\'d write to test this app end-to-end in Playwright

---

To test this app with Playwright: docker compose up sets the environment, npm run start:dev / npm run dev start the servers, then page.goto + expect assertions verify the UI. Mock GET /api/search with page.route() to test delay badges without burning AeroDataBox quota.